

## Day 32: Calculate Cart Total

### Day 32 — Calculate Cart Total

#### 1. Day Number

Day 32

#### 2. Topic Name

##### Cart Total Using Loop

Today you will learn how to:

- go through products stored in a cart,
  - read each product's price,
  - add those prices together,
  - store the result in a running total.
- 

#### 3. Connection

Yesterday, you learned how to **add a product to a cart**.

Conceptually, your cart looked something like:

```
cart
|
+-- Product 1
|
+-- Product 2
|
+-- Product 3
```

Today, you will calculate:

```
Product 1 price
    +
Product 2 price
    +
Product 3 price
    =
Cart Total
```

This introduces an important programming pattern called an **accumulator**.

---

#### 4. Important Topics

Today's main topics are:

- for loop
- accumulator variable
- dictionary access

- list of dictionaries

The key idea is:

```
Cart
↓
Loop through products
↓
Read price
↓
Add price to total
↓
Final cart total
```

## 5. Foundational Notes

### A. What is the cart?

A cart can be represented using a **list**.

Each item inside the list can be a **product dictionary**.

For example:

```
cart = [
    {"name": "Book", "price": 200},
    {"name": "Pen", "price": 50}
]
```

Here:

```
cart
|
+-- {"name": "Book", "price": 200}
|
+-- {"name": "Pen", "price": 50}
```

The cart contains **two dictionaries**.

### B. Accessing a product price

A dictionary stores information using **key-value pairs**.

Example:

```
product = {
    "name": "Book",
    "price": 200
}
```

You can access the price using:

```
product["price"]
```

Result:

```
200
```

So if a loop gives you one product at a time:

```
for product in cart:
```

you can access that product's price with:

```
product["price"]
```

---

### C. Why do we need a loop?

Suppose the cart contains:

```
Book → ₹200  
Pen   → ₹50  
Bag   → ₹500
```

You could manually write:

```
200 + 50 + 500
```

But a real cart may contain:

```
2 products  
10 products  
100 products
```

Therefore, we use a loop.

Conceptually:

```
for every product in cart:  
    read its price
```

---

### D. What is an accumulator?

An **accumulator** is a variable that keeps collecting values.

For example:

```
total = 0
```

Initially:

```
total = 0
```

Read first price:

```
price = 200

total = 0 + 200
total = 200
```

Read second price:

```
price = 50

total = 200 + 50
total = 250
```

So total keeps accumulating prices.

---

## E. Why start the total at 0?

Before processing the cart, nothing has been added yet.

Therefore:

```
total = 0
```

If the cart contains:

```
₹200
₹50
```

the calculation becomes:

```
Start
total = 0

Product 1
total = 0 + 200
      = 200

Product 2
total = 200 + 50
      = 250
```

Final result:

```
250
```

---

## 6. Easy Example

Consider some numbers:

```
numbers = [10, 20, 30]
```

You want to calculate their total.

Conceptually:

```
total = 0

10 → total becomes 10
20 → total becomes 30
30 → total becomes 60
```

The important pattern is:

```
initialize accumulator
    ↓
loop through values
    ↓
add current value
    ↓
keep updated total
```

For today's problem, instead of directly getting numbers, the numbers are stored inside product dictionaries.

```
product
    ↓
product["price"]
    ↓
add to total
```

---

## 7. Problem Statement

Create a **cart** containing **two product dictionaries**.

Each product should contain at least:

```
name
price
```

Example structure:

```
Cart
|
+-- Product 1
|   name
|   price
|
+-- Product 2
|   name
|   price
```

Use a **loop** to calculate the total price of all products in the cart.

Finally, print the cart total.

Do not manually add the two prices outside the loop.

---

## 8. Concepts Used

You will use four main concepts.

### List

Stores multiple products.

```
cart = [...]
```

### Dictionary

Stores information about one product.

```
{  
    "name": ...,  
    "price": ...  
}
```

### Loop

Processes each product one by one.

```
for product in cart:
```

### Accumulator

Keeps the running total.

Conceptually:

```
total = 0  
  
total = total + current_price
```

---

## 9. Thought Process

Before writing Python, think about the problem in small steps.

### Step 1 — What data do I have?

You have a cart.

The cart contains product dictionaries.

```
cart  
↓  
product  
↓  
name + price
```

## Step 2 — What information do I need?

To calculate the total, you mainly need:

```
price
```

The product name is not necessary for the calculation.

## Step 3 — Where will I store the running result?

Create a variable representing the total.

Initially:

```
total = 0
```

## Step 4 — How do I process every product?

Use a loop.

```
Cart
↓
Product 1
↓
Product 2
↓
...
```

## Step 5 — What happens for each product?

Read its price.

Then add that price to the running total.

```
current total
+
product price
↓
new total
```

## Step 6 — When should I print the final total?

After all products have been processed.

Be careful about the difference between:

```
inside loop → running total
outside loop → final total
```

---

## 10. Pseudocode

```
START
```

```
Create product 1 with:
```

```
name
price
```

Create product 2 with:

```
name
price
```

Create cart containing product 1 and product 2

Set total to 0

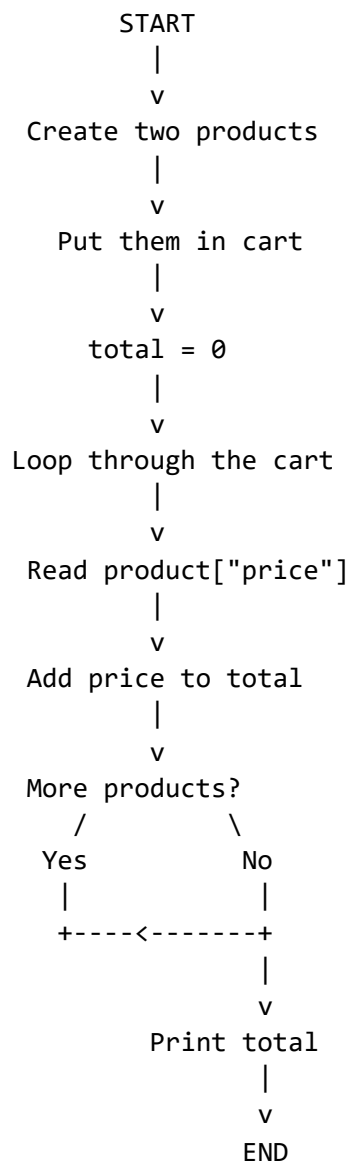
FOR each product in cart:

```
  get product price
  add product price to total
```

Print total

END

Example flow:



---

## 11. Suggested Solving Approach — Functional Approach

For today's exercise, use a simple function.

Think about creating a function whose responsibility is:

```
cart comes in
  ↓
function calculates total
  ↓
total comes out
```

Conceptually:

```
calculate_cart_total(cart)
```

Inside the function:

1. Create total
2. Loop through cart
3. Read each price
4. Add price to total
5. Return total

The caller can then print the returned value.

This is a good habit because the calculation logic stays separate from the rest of the program.

```
Cart data
  |
  v
calculate_cart_total()
  |
  v
Total
  |
  v
Print
```

---

## 12. Easy Edge Cases

### Edge Case 1 — Empty Cart

Suppose:

```
cart = []
```

There are no products.

The loop runs:

0 times

If your accumulator started at:

total = 0

then the final total naturally remains:

0

Expected idea:

```
Empty cart
  ↓
No products processed
  ↓
Total = 0
```

This shows why initializing the accumulator correctly is important.

---

## Edge Case 2 — Product Price Is 0

Suppose:

```
Product A → ₹100
Product B → ₹0
```

Calculation:

```
0 + 100 = 100
100 + 0 = 100
```

Final total:

100

A price of 0 should not cause an error.

---

## 13. Expected Input

For today's exercise, you do **not need** `input()` from the user.

You can create the data directly in Python.

Conceptual input:

```
Product 1:
name = "Book"
price = 200

Product 2:
```

```
name = "Pen"
price = 50
```

Cart:

```
[
    Book → 200,
    Pen  → 50
]
```

---

## 14. Expected Output

For the example above:

```
Book = ₹200
Pen  = ₹50
```

The calculated result should be:

```
Cart Total: 250
```

The important part is that 250 must be calculated using the **loop**, not by manually writing:

```
200 + 50
```

---

## 15. Hint Only

Start with an accumulator:

```
total = 0
```

Then think about:

```
for product in cart:
```

Inside that loop, ask yourself:

| How can I get the "price" value from the current product dictionary?

Then update:

```
total = old total + current product price
```

For the functional approach, your structure can begin like:

```
def calculate_cart_total(cart):
    # initialize total
    # loop through products
    # update total
    # return total
```

Your main learning pattern for **Day 32** is:

List of dictionaries

↓

Loop

↓

Dictionary access

↓

Price

↓

Accumulator

↓

Cart Total

**Do not use `sum()` for this exercise yet.** The goal today is to understand how a loop and accumulator work together.